SP

Sistema de Inventario Santa Patricia

Panadería · Pastelería · Cafetería · Minimarket — Sprint 1 (HU01 y HU02)

Documento del proyecto: qué hace, cómo se instala, cómo se usa y qué cumple

HISTORIAS	SUBTAREAS	PRUEBAS	DATOS
2 / 2	**21**	**18 / 18**	**51**
COMPLETADAS	REVISADAS	PASAN	PRODUCTOS · 6 USUARIOS

Contenido

1. Qué es y qué hace el sistema
2. Estructura y arquitectura
3. Base de datos: tablas y campos
4. Cómo leer la pantalla del inventario
5. Formulario «Nuevo producto» y «¿Cómo llega?»
6. HU01 · Acceso y roles: subtareas y por qué cumple
7. HU02 · Registrar insumo: subtareas y por qué cumple
8. La regla del profesor: nada se toca a mano en la base
9. Instalación en la PC de cada compañero
10. Usuarios de prueba
11. Avance del proyecto
12. Pendientes y observaciones (incluye «assets»)
13. Preguntas frecuentes

1. Qué es y qué hace el sistema

Es una aplicación web para controlar el **catálogo de insumos y productos** de Santa Patricia. En este Sprint 1 permite:

Función	Qué hace
Iniciar sesión (HU01)	Cada trabajador entra con su correo y contraseña. Según su **rol** (Administrador, Encargado de Producción, Cajero o Almacenero) puede hacer más o menos cosas.
Seguridad	Si alguien se equivoca de contraseña 5 veces seguidas, la cuenta se bloquea 15 minutos. Sin iniciar sesión no se puede ver nada.
Ver el inventario	Tabla con todos los productos: código, nombre, área, stock, precio, vencimiento y estado (Óptimo, Bajo o Crítico). Tarjetas con el resumen.
Buscar y filtrar	Buscador por nombre o código (encuentra «azúcar» aunque se escriba sin tilde) y botones para ver solo un área (Cocina, Panadería, Pastelería, Cafetería, Minimarket).
Registrar productos (HU02)	Solo el Administrador. Formulario con nombre, área, unidad, stock, stock mínimo, precio y vencimiento. El **código se genera solo** (Azúcar → AZU-001). El sistema calcula el stock si llega en sacos o cajas.
Eliminar productos	Solo el Administrador. El producto deja de aparecer, pero **no se borra** de la base (se guarda el historial).

> **¿Cómo se abre?**
> En NetBeans se abre la carpeta `backend` y se pulsa **Run**. Se abre solo el navegador en el login: `http://localhost:8087/santapatricia/` . No hace falta usar la terminal.

2. Estructura y arquitectura

Sigue el esquema del documento del curso (puntos 1, 2 y 3): **React → API REST con JSON → Java con Servlets → MySQL**.

FRONTEND · React (carpeta `frontend/`) — pantallas: login, inventario, formulario. Hecho por el equipo de front.

▼ petición HTTP con JSON (GET, POST, PUT, DELETE) + token

SERVLET (`servlet/`) — recibe la petición y responde. No tiene reglas de negocio.

▼

SERVICE / SERVICEIMPL (`service/`) — reglas del negocio: validar datos, generar el código, bloquear tras 5 intentos.

▼

DAO / DAOIMPL (`dao/`) — el único lugar donde hay SQL. Usa JDBC.

▼

MySQL · santapatricia_db (`database/`) — tablas, relaciones y claves.

Carpetas del proyecto

```
SantaPatricia/
├── frontend/                 React + TypeScript + Tailwind (VS Code)
│   └── src/  pages/ components/ services/ (llamadas a la API) types/
├── backend/                  Proyecto Maven (NetBeans) → GlassFish 8
│   ├── src/main/java/pe/edu/isil/santapatricia/
│   │   ├── model/            Usuario, Producto, Categoria (representan las tablas)
│   │   ├── dto/              JSON que entra y sale (LoginRequest, ProductoDTO…)
│   │   ├── servlet/          AuthServlet, ProductoServlet, CatalogoServlet, SaludServlet
│   │   ├── service/ + impl/  AuthServiceImpl, ProductoServiceImpl (reglas)
│   │   ├── dao/ + impl/      UsuarioDAOImpl, ProductoDAOImpl, CategoriaDAOImpl (SQL)
│   │   └── config/           conexión MySQL, JSON, token JWT, filtros de seguridad y CORS
│   ├── src/main/resources/config.properties   ← contraseña de MySQL de cada uno
│   └── src/main/webapp/app/  ← el React ya compilado (por eso Run abre todo)
├── database/
│   ├── esquema.sql           crea la base y las 4 tablas
│   └── datos_prueba.sql      6 usuarios, 5 áreas y 51 productos
└── README.md
```

Tecnologías

Parte	Tecnología	Para qué
IDE	NetBeans + VS Code	Backend en NetBeans; diseño del front en VS Code
Frontend	React, TypeScript, Tailwind, Axios	Pantallas y llamadas a la API
Backend	Java 21, Servlets (Jakarta EE 11)	Recibir peticiones y aplicar reglas
Acceso a datos	JDBC	SQL directo, fácil de entender

Servidor	GlassFish 8	Ejecuta el backend y entrega el front
Base de datos	MySQL 8	Guardar la información
Sesión	JWT (token)	Saber quién hace cada petición y con qué rol
Pruebas	JUnit	18 pruebas automáticas
Gestión	Maven, Git, Jira	Dependencias, versiones y tareas

3. Base de datos: tablas y campos

Base `santapatricia_db`, 4 tablas. Los campos marcados `nuevo` no existían en la versión anterior del proyecto.

Tabla `productos` (el catálogo, HU02)

Campo	Tipo	Para qué sirve
`id`	número	Número interno que pone MySQL solo. No lo ve el usuario.
`sku` `nuevo`	texto (20)	**Código del producto**. Lo genera el sistema: 3 primeras letras del nombre + número. «Azúcar rubia» → **AZU-002**. No se repite.
`nombre`	texto (100)	Nombre del producto. No se puede repetir (sin importar mayúsculas).
`categoria_id`	número	El **área** a la que pertenece (enlace a la tabla categorias).
`unidad_medida`	KG, G, L, ML, UND, PAQ	Cómo se cuenta ese producto: kilos, gramos, litros, mililitros, unidades o paquetes.
`stock_actual` `nuevo`	decimal	**Cuánto hay ahora** en el almacén, en la unidad de medida.
`stock_minimo`	decimal	**Cuánto debería haber como mínimo**. Si el stock baja de aquí, hay que comprar.
`precio_unitario` `nuevo`	decimal	Precio en soles **por unidad de medida** (por KG, por L...), no por saco.
`fecha_vencimiento` `nuevo`	fecha	Cuándo vence. Vacío = no vence (bolsas, vasos, detergente).
`activo`	sí / no	«No» = eliminado. El producto no se borra, solo se oculta.
`creado_en`, `actualizado_en`	fecha y hora	Cuándo se registró y cuándo cambió por última vez (los pone MySQL solo).

> **El «estado» no se guarda**
> Óptimo / Bajo / Crítico **se calcula** cada vez comparando el stock actual con el mínimo (ver sección 4). Así nunca queda desactualizado.

Tabla `usuarios` (HU01)

Campo	Para qué sirve
`nombre`, `correo`	Datos del trabajador. El correo es único y es con lo que inicia sesión.
`password`	Contraseña. **Por decisión del equipo se guarda visible** para entenderla y probarla (ver sección 12).
`rol_id`	Su rol (enlace a la tabla roles).
`activo`	«No» = dado de baja: no puede entrar.

`intentos_fallidos`	Cuántas contraseñas incorrectas seguidas lleva. Vuelve a 0 al entrar bien.
`bloqueado_hasta`	Si tiene una hora, la cuenta está bloqueada hasta esa hora (5 fallos → 15 minutos).
`ultimo_acceso`	Última vez que entró correctamente.

Tablas `roles` y `categorias`

roles (código)	Se muestra como
`ADMINISTRADOR`	Administrador
`ENCARGADO_PRODUCCION`	Encargado de Producción
`CAJERO`	Cajero
`ALMACENERO`	Almacenero

categorias (código)	Área
`COCINA`	Cocina
`PANADERIA`	Panadería
`PASTELERIA`	Pastelería
`CAFETERIA`	Cafetería
`MINIMARKET`	Minimarket

4. Cómo leer la pantalla del inventario

¿Qué significa «45 / mín. 20 L»?

> **45** / mín. 20 L
>
> **45** = **stock actual**: hay 45 litros de ese producto ahora en el almacén.
> **mín. 20** = **stock mínimo**: lo que siempre debería haber; si baja de 20, hay que comprar.
> **L** = la **unidad de medida** del producto (litros).
>
> Ejemplo real de la base: **Aceite vegetal** → **45 / mín. 20 L**: hay 45 litros y el mínimo es 20, está bien abastecido.

El estado del stock (semáforo)

Estado	Regla	Ejemplo (mínimo = 20)
Óptimo	El stock es **mayor** que el mínimo	Hay 45 → 45 > 20 → **Óptimo**
Stock bajo	El stock es **igual o menor** que el mínimo	Hay 15 → 15 ≤ 20 → **Bajo**: conviene comprar pronto
Crítico	El stock es **la mitad del mínimo o menos**	Hay 8 → 8 ≤ 10 (mitad de 20) → **Crítico**: comprar ya

Las otras columnas

Columna	Qué muestra
SKU	El código automático del producto (ej. `ACE-001`).
Precio	«S/ 9.90 / L» = cada litro cuesta 9.90 soles.
Vencimiento	Fecha en día/mes/año. «—» = no vence.
Tarjetas de arriba	Total de productos, cuántos están Óptimos, en Stock bajo y Críticos. Siempre suman el total.
Botones de área	«Pastelería (14)» = hay 14 productos de pastelería; al pulsarlo solo se ven esos.
Buscador (arriba)	Busca por nombre o código mientras escribes.
Papelera	Eliminar. Solo la ve el Administrador.

5. Formulario «Nuevo producto» y «¿Cómo llega?»

Solo el **Administrador** ve el botón **Nuevo producto**. Los campos con * son obligatorios.

Campo	Qué se escribe
Nombre *	Qué producto es. Ej.: «Azúcar rubia». Debajo aparece el **código que se asignará** (AZU-005).
Área *	Cocina, Panadería, Pastelería, Cafetería o Minimarket.
Unidad de medida *	En qué se va a **contar** el stock: KG, G, L, ML, UND (unidad) o PAQ (paquete).
¿Cómo llega? (opcional)	Calculadora para cuando la mercadería llega en envases (ver abajo).
Stock que entra *	La cantidad total que entra, en la unidad elegida. Se llena sola si usas «¿Cómo llega?».
Stock mínimo *	Debajo de esta cantidad el producto se marca como Bajo o Crítico.
Precio (S/) *	Precio **por unidad de medida**: por KG, por L, por UND… (no por saco).
Vencimiento	Fecha de vencimiento. Vacío = no vence.

«¿Cómo llega?» explicado

> **[cantidad de envases]** sacos / cajas / paquetes de **[contenido de cada envase] [unidad]** = **stock que entra**
>
> **Primer número** = **cuántos envases llegaron** (sacos, cajas, jabas, paquetes…).
> **Segundo número** = **cuánto trae cada envase**, medido en la unidad elegida.
> El sistema los **multiplica** y llena «Stock que entra».

Llega a la panadería…	Unidad elegida	Se escribe	Stock que entra
10 sacos de azúcar de 10 kg	KG	10 sacos de 10	**100 KG**
5 sacos de harina de 50 kg	KG	5 sacos de 50	**250 KG**
3 jabas de huevos de 30	UND	3 jabas de 30	**90 UND**
2 cajas de leche con 12 litros	L	2 cajas de 12	**24 L**
4 cajas de 500 g de polvo de hornear	G	4 cajas de 500	**2000 G**

> **Importante**
> «¿Cómo llega?» **solo es una calculadora**: en la base se guarda únicamente el total (100 KG), no los sacos. Si la mercadería no llega en envases, se deja vacío y se escribe directo el stock que entra. El precio siempre es por la unidad (por KG), no por el saco.

Qué pasa al pulsar Guardar

1. React envía los datos al backend (`POST /api/inventario/productos`) con el token del usuario.

2. El backend revisa que el usuario sea **Administrador**; si no, responde «Su rol no tiene permiso».
3. Revisa cada campo: vacíos, negativos, unidad o área inexistente. Si algo falla, el mensaje aparece **debajo de cada casilla**.
4. Revisa que el nombre no exista ya («Ya existe un producto llamado…»).
5. Genera el código (AZU-005), guarda en MySQL y devuelve el producto.
6. La pantalla muestra un aviso verde y el producto **resaltado en verde** en la lista.

6. HU01 · Control de acceso y autenticación por roles (SISPS-2)

> **Criterio de aceptación (Jira)**
> Dado que ingreso credenciales válidas de un usuario «Cajero/Almacenero», cuando el sistema valide la autenticación y genere el token (JWT), entonces se debe redirigir a la vista de inventario restringiendo el acceso a configuraciones de usuarios o edición de parámetros del negocio.

✅ **Cumple el criterio:** el cajero inicia sesión, recibe su token, entra al Inventario Central y **no ve** el botón de registrar ni el de eliminar, y ve un aviso de que su rol solo puede consultar. Si intenta registrar directamente por la API, el backend responde **403 (sin permiso)**.

	Subtarea	Por qué cumple	Cómo funciona / cómo se comprueba
✅	**SISPS-13** Crear BD y tabla de usuarios con usuarios de prueba	Existen las tablas `roles` y `usuarios` con 6 usuarios de prueba.	Ejecutar `esquema.sql` y `datos_prueba.sql` ; al final muestran los usuarios.
✅	**SISPS-14** Clase de conexión (JDBC a MySQL)	`ConexionBD` abre conexiones JDBC; los datos salen de `config.properties` , no están escritos en el código.	La página `estado.html` muestra «conectada a santapatricia_db».
✅	**SISPS-15** Modelo Usuario y Rol	Clase `Usuario` con su rol (código y nombre) y los 4 roles de Jira.	El login devuelve el rol: ADMINISTRADOR, CAJERO…
✅	**SISPS-16** DAO: buscar por correo, registrar intento fallido y acceso	`UsuarioDAOImpl` : `buscarPorCorreo` , `registrarIntentoFallido` , `registrarAccesoExitoso` .	Tras un fallo, `intentos_fallidos` sube; al entrar bien vuelve a 0 y se guarda `ultimo_acceso` .
⚠️	**SISPS-17** AuthService: validar credenciales con BCrypt y bloqueo tras 5 intentos	Valida credenciales y bloquea tras 5 intentos ✅. **BCrypt no se usa**: el equipo decidió guardar las contraseñas visibles para entenderlas.	Prueba automática «5 contraseñas incorrectas bloquean la cuenta». Ver sección 12 sobre las contraseñas.
✅	**SISPS-18** Login y Logout con redirección por rol	`POST /api/auth/login` entrega el token y el rol; React redirige al inventario. Cerrar sesión borra el token.	Entrar con cualquiera de los usuarios; el ícono de salida vuelve al login.
✅	**SISPS-19** Filtro que protege las rutas	`SeguridadFilter` revisa el token en toda la API; sin token, o con uno falso o vencido → 401.	Entrar a `/api/inventario/productos` sin iniciar sesión → «Su sesión expiró o no ha iniciado sesión».
✅	**SISPS-20** Vistas: login, panel, inventario, acceso denegado	Pantallas en React (equipo de front): login e Inventario Central; aviso para roles sin permiso.	Se abren con Run en NetBeans.
✅	**SISPS-21** Que el bloqueo no deje al admin sin acceso	El bloqueo es **temporal (15 min)** y no da de baja la cuenta.	Prueba automática: tras el bloqueo la cuenta sigue activa.
✅	**SISPS-22** Pruebas JUnit de AuthService	**7 pruebas** en `AuthServiceImplTest` , todas pasan.	NetBeans → clic derecho en backend → Test.

✅	**SISPS-23** Criterio de aceptación con JWT	La sesión usa token JWT, como dice el criterio, para que React pueda conectarse.	El login devuelve `{ token, usuario }`.
✅	**SISPS-33** CORS para que React consuma la API	`CorsFilter` permite el modo diseño (localhost:5173).	`npm run dev` en VS Code se conecta sin errores.

7. HU02 · Registrar insumo base (SISPS-3)

> **Criterio de aceptación (Jira)**
> Dado que me encuentro autenticado en el módulo de catálogo, cuando complete los campos (Nombre «Harina Especial», Categoría, Unidad «Kg», Stock mínimo «50») y presione «Guardar», entonces el sistema debe almacenar el registro, validar que el código no esté duplicado y reflejar el nuevo elemento en la lista general de productos.

✅ **Cumple el criterio:** el administrador completa el formulario, pulsa Guardar, el producto se guarda en MySQL, el código se genera solo y nunca se repite, y el producto aparece **resaltado** en la lista. Si el nombre ya existe o falta un dato, el sistema lo indica.

	Subtarea	Por qué cumple	Cómo funciona / cómo se comprueba
✅	**SISPS-24** Tabla con código único, nombre, categoría, unidad, stock mínimo y stock actual	Tabla `productos` con todos esos campos; `sku` y `nombre` son únicos.	Ver sección 3. MySQL rechaza códigos repetidos.
✅	**SISPS-25** Modelo Producto y Categoría	Clases `Producto` y `Categoria`; las áreas están en una tabla (se pueden agregar sin programar).	Las 5 áreas aparecen en el formulario y en los filtros.
✅	**SISPS-26** DAO: insertar, listar, buscar por código	`ProductoDAOImpl`: `insertar`, `listarActivos`, `existeSku` y `ultimoNumeroSku` (para generar el siguiente código).	La búsqueda por código se hace en la pantalla, con el buscador.
✅	**SISPS-27** Servicio: validar campos obligatorios y código duplicado	`ProductoServiceImpl` revisa cada campo y el nombre repetido; el código nunca se repite porque se genera con el siguiente número libre.	Guardar vacío → 6 mensajes, uno por casilla. Nombre repetido → aviso.
✅	**SISPS-28** Servlet: mostrar formulario, guardar y listar	`ProductoServlet` (listar, guardar, ver, editar, eliminar) y `CatalogoServlet` (opciones del formulario).	Ver la tabla de la API en el README.
✅	**SISPS-29** Vista: formulario de registro y tabla del catálogo	Formulario «Nuevo producto» (estaba pendiente en el front) y tabla con precio, vencimiento y estado.	Entrar como admin → Nuevo producto.
✅	**SISPS-30** Restringir el registro al Administrador	El backend responde 403 a los otros roles y la pantalla les oculta el botón.	Entrar como `caja@…`: no aparece el botón.
✅	**SISPS-31** Datos de prueba reales de Santa Patricia	51 productos: harinas, levaduras, azúcares, lácteos, café, empaques… en las 5 áreas.	Ejecutar `datos_prueba.sql`.

✅	**SISPS-32** Pruebas: código duplicado y campos vacíos	**11 pruebas** en `ProductoServiceImplTest` : código automático, campos vacíos, datos inválidos, nombre y código repetidos.	NetBeans → Test. Las 18 pasan.

8. La regla del profesor: nada se toca a mano en la base

La regla es que las entradas, salidas y cualquier dato **no se escriban directamente en MySQL**, sino siempre a través del sistema. Este proyecto la cumple así:

	Cómo se cumple	Explicación
✅	Todo dato nuevo entra por la pantalla	El usuario registra productos en el formulario; nadie necesita abrir Workbench para agregar o cambiar datos.
✅	Un solo camino hacia la base	React → Servlet → Service → DAO → MySQL. **El único código que escribe SQL es el DAO**; las pantallas nunca tocan la base.
✅	Reglas antes de guardar	El Service valida cada dato (vacíos, negativos, repetidos) **antes** de que llegue a MySQL. Un dato mal escrito no se guarda.
✅	Solo quien tiene permiso	Solo el Administrador registra o elimina, y el backend lo verifica con el token en cada petición.
✅	Nada se borra	Eliminar = desactivar (`activo = 0`). La información queda para el historial y las auditorías.
✅	Los scripts SQL son solo para instalar	`esquema.sql` y `datos_prueba.sql` se usan **una vez**, para crear la base y cargar los datos iniciales. Después todo se hace desde el sistema.
✅	Consultas protegidas	Se usa `PreparedStatement` con «?»: lo que escribe el usuario nunca se mezcla con el SQL (evita la inyección SQL).

> **A tener en cuenta para el Sprint 2**
>
> En el Sprint 1 el stock se escribe al **registrar** el producto. Las entradas por compra (HU03) y las salidas por venta o producción (HU04, HU05) aún no existen. Cuando se hagan, el stock debe cambiar **solo con movimientos registrados** (con fecha, usuario y motivo, para el kárdex). Hoy la API permite editar un producto, incluido su stock (`PUT` , solo Administrador, sin pantalla todavía). Conviene que en el Sprint 2 la edición ya no cambie el stock y que este solo se mueva con entradas y salidas.

9. Instalación en la PC de cada compañero

Qué necesita cada uno

Programa	Para qué	Nota
MySQL 8 + Workbench	Base de datos	Recordar la contraseña de su usuario root
Apache NetBeans	Abrir y ejecutar el backend	Trae Maven incluido
GlassFish 8 registrado en NetBeans	Servidor	*Tools → Servers → Add Server → GlassFish Server*
JDK 21	Java	JDK 25 también sirve
Node.js + VS Code	Solo para quien cambie el diseño del front	Para usar el sistema no hace falta

Pasos

1. **Copiar la carpeta** `SantaPatricia` (ZIP o GitHub) a una ruta sin espacios ni tildes, por ejemplo `C:\Proyecto\SantaPatricia`.
2. **Crear la base:** en MySQL Workbench, *File → Open SQL Script →* `database\esquema.sql` → rayo ⚡. Luego lo mismo con `database\datos_prueba.sql`. Deben verse 6 usuarios y 51 productos.
3. **Poner su contraseña de MySQL:** abrir `backend\src\main\resources\config.properties` y cambiar `db.password=root` por la suya.
4. **Abrir en NetBeans:** *File → Open Project →* carpeta `backend` (ícono de mundo).
5. **Pulsar Run ▶.** La primera vez tarda unos minutos (descarga librerías). Se abre el navegador en el login.
6. **Entrar** con `admin@santapatricia.pe` / `Admin123`.

> **Sobre el puerto**
> En esta PC GlassFish usa el puerto **8087**. En otra PC puede ser **8080** (el normal de GlassFish). **No hay que cambiar nada**: NetBeans abre la dirección correcta solo y el sistema funciona en cualquier puerto.

Si algo falla

Lo que aparece	Solución
`estado.html` dice «base de datos SIN CONEXIÓN»	MySQL apagado, faltan los scripts del paso 2 o la contraseña del paso 3 no es la suya.
El login dice «No se pudo conectar con el servidor»	El backend no está corriendo: pulsar Run en NetBeans.
NetBeans pide elegir un servidor	Elegir GlassFish 8. Si no aparece, registrarlo (tabla de arriba).
NetBeans: `PKIX path building failed`	Antivirus que revisa HTTPS (Norton). *Tools → Options → Java → Maven → Global Execution Options*: `-Djavax.net.ssl.trustStoreType=Windows-ROOT`

`localhost:5173` «rechazó la conexión»	Esa dirección es solo para el modo diseño (`npm run dev`). Para usar el sistema: Run en NetBeans.
Un usuario «no puede entrar» con la contraseña correcta	Se bloqueó por 5 intentos fallidos: esperar 15 minutos o volver a ejecutar los scripts del paso 2.

Para el equipo de front (cambiar el diseño)

1. Abrir la carpeta `frontend` en VS Code → *Terminal* → *New Terminal*.
2. `npm run dev` para ver los cambios en vivo (localhost:5173). El backend debe estar corriendo.
3. Al terminar: `npm run build` . Copia la versión final dentro del backend. Luego Run en NetBeans.

10. Usuarios de prueba

Nombre	Correo	Contraseña	Rol	Puede
Valeria Ramos	`admin@santapatricia.pe`	`Admin123`	Administrador	Todo: ver, registrar, eliminar
Carlos Mendoza	`produccion@santapatricia.pe`	`Prod123`	Encargado de Producción	Ver inventario
Lucía Torres	`caja@santapatricia.pe`	`Caja123`	Cajero	Ver inventario
Jorge Quispe	`almacen@santapatricia.pe`	`Alma123`	Almacenero	Ver inventario
Rosa Huamán	`rosa.huaman@santapatricia.pe`	`Rosa123`	Cajero	Ver inventario
Pedro Salas	`pedro.salas@santapatricia.pe`	`Pedro123`	Almacenero	**Dado de baja**: no puede entrar

La contraseña distingue mayúsculas: `Admin123` funciona y `admin123` no.

11. Avance del proyecto

	Sprint / Historia	Estado
✅	Sprint 0 · Planificación y arquitectura	Documento del curso, Jira, estructura de carpetas según el PDF.
✅	Sprint 1 · HU01 Acceso y roles	Completa y probada (falta la validación del Product Owner en el Sprint Review).
✅	Sprint 1 · HU02 Registrar insumo	Completa y probada (falta la validación del Product Owner).
🟡	Sprint 2 · HU03 Entradas, HU04 Salidas, HU05 Producción	Por hacer. La base ya tiene stock actual para que estos movimientos lo sumen o resten.
🟡	Sprint 3 · HU06 Mermas, HU07 Alertas, HU08 Kárdex, HU09 Ajustes	Por hacer. El estado (Bajo/Crítico) ya sirve de base para las alertas.
🟡	Sprint 4 · HU10 Dashboard (solo Administrador)	Por hacer.

Verificaciones hechas: 18 pruebas automáticas; login correcto e incorrecto; usuario dado de baja; bloqueo; sesión que se mantiene al recargar; registro con código automático; nombre repetido; formulario vacío; permisos del cajero (sin botones y 403 en la API); eliminación lógica; buscador y filtros por área.

12. Pendientes y observaciones

«Error» en la carpeta assets

> **Qué es:** NetBeans marca en rojo los archivos de `backend/src/main/webapp/app/assets` (`index-….js` y `index-….css`).
>
> **Por qué pasa:** esos archivos **no se escriben a mano**: los genera `npm run build` con todo el código de React comprimido en una sola línea, y con reglas modernas de CSS (Tailwind 4: `@property`, `@layer`) que el revisor de NetBeans no conoce. NetBeans los revisa como si fueran código normal y los marca.
>
> **¿Hace algo malo? No.** En el navegador esos archivos cargan correctamente (sin errores en la consola) y el sistema funciona.
>
> **Cómo se corregirá (más adelante):** que NetBeans ignore esa carpeta (es código generado) o servir el front compilado desde fuera de `src`. **No se tocó**, como se pidió.

Otras observaciones

Tema	Detalle
Contraseñas visibles	Decisión del equipo para entender y probar. En un sistema real se guarda un **hash** (por ejemplo BCrypt): un texto que no se puede revertir, para que, si alguien roba la base, no pueda leer las contraseñas. Jira (SISPS-2 y SISPS-17) todavía menciona BCrypt: hay que actualizarlo o volver a BCrypt antes de entregar.
Clave del token en config.properties	Sirve para trabajar en las PC del equipo. En producción debería ir en una variable de entorno.
Editar productos	La API ya permite editar (solo Administrador), pero aún no hay pantalla. Ver la nota de la sección 8 sobre el stock.
Cambios del front	Después de cambiar el diseño hay que ejecutar `npm run build`; si no, Run muestra la versión anterior.
Bootstrap	El documento del curso menciona Bootstrap; el equipo de front usa Tailwind (cumple la misma función).

13. Preguntas frecuentes

Pregunta	Respuesta
¿Qué significa «45 / mín. 20 L»?	Hay 45 litros (stock actual) y lo mínimo que debería haber son 20 litros. Ver sección 4.
¿Qué pongo en «¿Cómo llega?»?	Primer número: cuántos envases (sacos, cajas, jabas). Segundo: cuánto trae cada uno en la unidad elegida. El sistema multiplica. Es opcional.
¿Por qué no escribo el código del producto?	Lo genera el sistema con las 3 primeras letras del nombre + un número, para que siempre sea ordenado y nunca se repita.
¿Por qué el cajero no ve «Nuevo producto»?	Por HU01: solo el Administrador registra o elimina. Los demás roles consultan.
¿Por qué al eliminar no se borra de la base?	Para conservar el historial; en los próximos sprints las entradas, salidas y el kárdex harán referencia a esos productos.
¿El precio es por saco o por kilo?	Por la unidad de medida (por kilo si la unidad es KG).
¿Por qué el estado no se guarda?	Se calcula al momento comparando stock y mínimo; así nunca queda desactualizado.
¿Qué es el token (JWT)?	Un «pase» firmado que el backend entrega al iniciar sesión. React lo envía en cada petición; el backend sabe así quién es y qué rol tiene. Si alguien lo modifica, se rechaza.
¿Puedo volver a los datos iniciales?	Sí: ejecutar otra vez `esquema.sql` y `datos_prueba.sql`. Ojo: borra lo registrado.
¿Qué pasa si escribo mal la contraseña 5 veces?	La cuenta se bloquea 15 minutos. Es una medida de seguridad contra quien intenta adivinar contraseñas.